

SMART STUDENT TASK & PRODUCTIVITY TRACKER

Java Command-Line Application
Project Report

Student Name	Jayesh Karmodia
Registration No.	24BCY10075
Course	Programming in Java
Project Domain	Student Productivity & Task Management
Submission Date	17/09/2026

VIT Bhopal University

A Java project demonstrating object-oriented programming, database connectivity, validation, testing, and CLI application design.

1. Problem Statement & Relevance

Students commonly manage assignments, deadlines, and study records in separate notes or applications. This can make it difficult to identify urgent work, keep academic tasks organized, and measure study time. The Smart Student Task & Productivity Tracker provides one lightweight command-line application for these activities. It stores tasks persistently, tracks study sessions, shows productivity information, and recommends a pending task using a transparent score.

Why it matters:

The project combines task management with study tracking instead of acting only as a basic to-do list. The student can view pending and overdue work, record study time, check completion information, and use the recommendation feature to identify a task to work on next.

2. Relevance to Course Concepts

- **Object-Oriented Programming:** The project is separated into model classes, service classes, database access, UI, and utility classes. This demonstrates encapsulation, class design, constructors, methods, and separation of responsibilities.
- **Collections Framework:** Java List and stream operations are used to store, filter, sort, and recommend pending tasks.
- **Exception Handling:** SQL, file, numeric, and invalid-input errors are handled so that the CLI can continue running and display useful error messages.
- **Date and Time API:** LocalDate and LocalDateTime are used for deadlines and study-session timing.
- **JDBC and SQLite:** Java database connectivity is used to create tables, insert records, retrieve records, update status, and calculate productivity data.
- **Testing:** JUnit 5 is used to verify task overdue logic and smart recommendation behavior.

3. Code Organization & Structure

The project follows a separation-of-concerns approach so that each part of the application has a clear responsibility.

- **Task.java, Student.java, StudySession.java:** Model classes that represent the main data used by the application.
- **Priority.java and TaskStatus.java:** Enums used for task priority and task state.
- **TaskService.java:** Handles task creation, retrieval, completion, and deletion using the database.
- **SchedulerService.java:** Calculates recommendation scores and selects the highest-scoring pending task.
- **StudySessionService.java:** Stores and retrieves study sessions and calculates total study time.
- **ProductivityService.java:** Calculates task counts, overdue tasks, and subject-wise study time.
- **BackupService.java:** Creates timestamped copies of the SQLite database.
- **DatabaseManager.java:** Opens the JDBC connection, creates the SQLite tables, and closes the connection.
- **ConsoleUI.java and Main.java:** Provide the command-line menu and application entry point.
- **InputValidator.java and DateUtil.java:** Provide validation and date-related helper methods.

4. Development Process & Version Control

The project was maintained using GitHub for source-code and documentation version control. The repository contains the Java source, Maven configuration, README, statement, diagrams, and project report. The final local project was built and tested with Maven before the updated documentation and design artifacts were pushed to the public repository.

5. Key Decisions & Approach

- **SQLite persistence:** Local database storage was chosen so task and study-session information remains available across application runs without requiring a separate server.
- **Rule-based recommendation:** A transparent scoring formula combines priority, deadline urgency, estimated effort, and an overdue bonus.
- **Unique database IDs:** Each task and study session receives an auto-incremented ID, making task operations straightforward.
- **CLI-first design:** A command-line interface keeps the project simple to run and suitable for a clean terminal environment.

6. Challenges Faced & Solutions

- **User input errors:** Invalid dates and numeric values can cause runtime problems. The project uses `InputValidator` and exception handling to reject invalid input and show clear messages.
- **Database persistence:** Data had to be saved and retrieved correctly between sessions. `DatabaseManager` initializes the SQLite schema, while service classes use prepared statements for database operations.
- **Recommendation scoring:** Several task factors must be combined into one score. `SchedulerService` assigns weights for priority, deadline urgency, effort, and overdue status and then selects the highest-scoring incomplete task.
- **Backup handling:** Backups must be created without changing the main database. `BackupService` creates a backups directory and copies the database using a timestamped filename.
- **Testing:** Core behavior needed verification before submission. JUnit tests were added for overdue detection and recommendation behavior, and the Maven test run completed successfully.

7. Testing & Verification

The project was verified with Maven using the command **`mvn clean test`**. The development test run reported 4 tests with 0 failures, 0 errors, and 0 skipped. The command-line application was also started manually, and invalid date input was correctly rejected with a validation message.

8. What I Learned

- How to structure a Java application using separate model, service, UI, database, and utility classes.
- How JDBC can be used with SQLite for persistent application data.
- How Java Date/Time classes can be used for deadlines and study sessions.
- How validation and exception handling improve CLI reliability.
- How Maven and JUnit can be used for repeatable build and testing.
- How GitHub can be used to maintain and document a complete academic project.

9. Conclusion

The Smart Student Task & Productivity Tracker is a Java command-line application that brings together academic task management, deadline tracking, study-session recording, productivity information, smart task recommendation, and database backup in one system.

The implementation demonstrates core Java concepts through object-oriented classes, enums, collections, exception handling, date and time handling, JDBC-based SQLite persistence, and JUnit testing. The layered organization also keeps the code easier to understand and maintain.

The completed project provides a practical terminal-based solution for student task and study management while demonstrating the Java programming concepts covered in the course.

10. Future Scope

- Task editing and search functionality.
- Recurring tasks and configurable priorities.
- Export of productivity data to CSV.
- Authentication and multiple student profiles.
- Optional GUI or web-based interface.
- Optional external reminders and notifications.

Project Repository

<https://github.com/jayeshkarmodia/SmartStudentTracker>

Submitted by: Jayesh Karmodia | Reg. No.: 24BCY10075